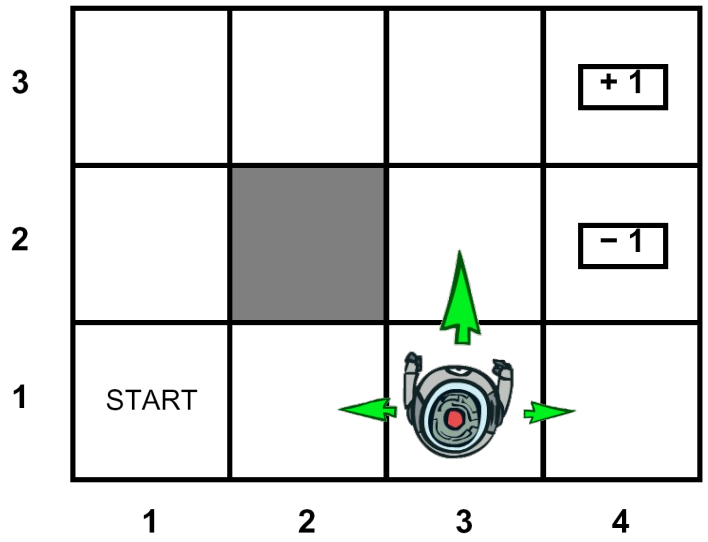
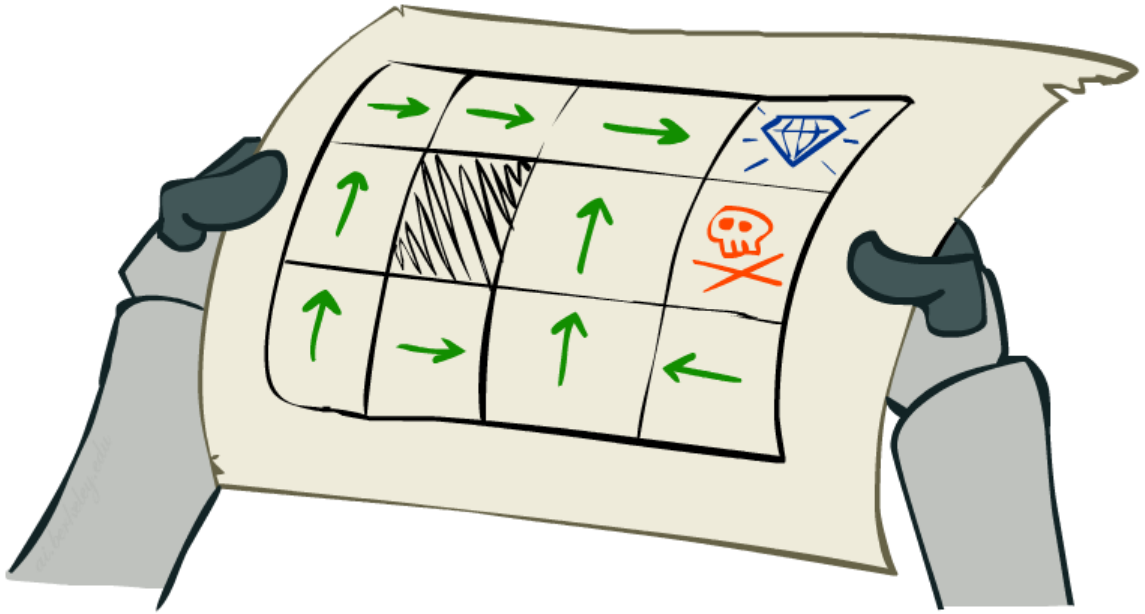
Recap - Markov Decision Processes - Elements

- An MDP is defined by:
  - A set of states s -S
  - A set of actions a -A
  - A transition function $T(s, a, s')$
    - Probability that a from s leads to s', i.e., $P(s' | s, a)$
    - Also called the model or the dynamics
  - A reward function $R(s, a, s')$
    - Sometimes just $R(s)$ or $R(s')$
  - A start state
  - Maybe a terminal state
- Policies
- Stationary Preferences

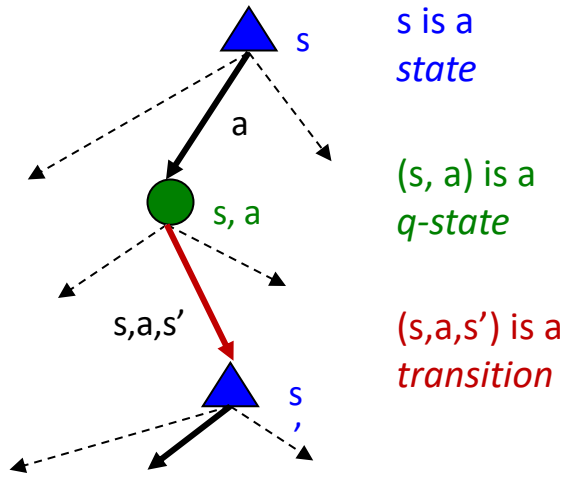


Solving MDPs

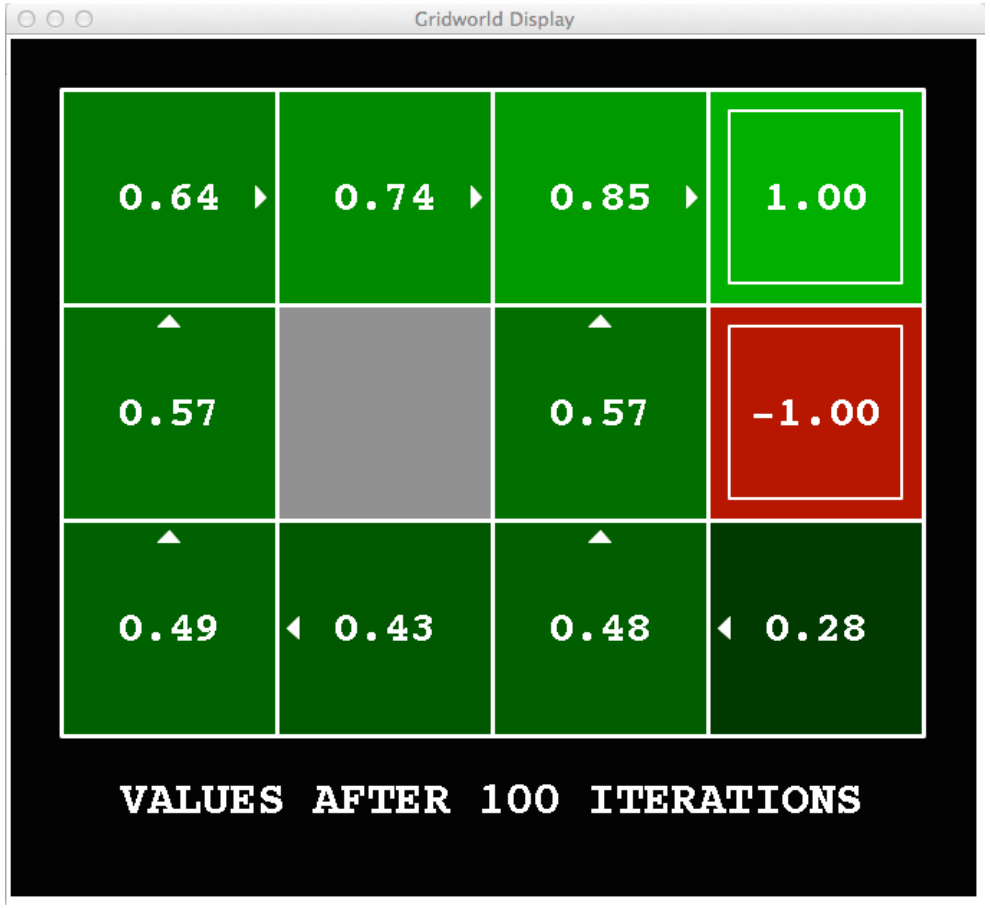


Optimal Quantities

- The value (utility) of a state s:
  - $V^*(s)$ = expected utility starting in s and acting optimally
- The value (utility) of a q-state (s, a):
  - $Q^*(s, a)$ = expected utility starting out having taken action a from state s and (thereafter) acting optimally
- The optimal policy:
  - $\pi^*(s)$ = optimal action from state s

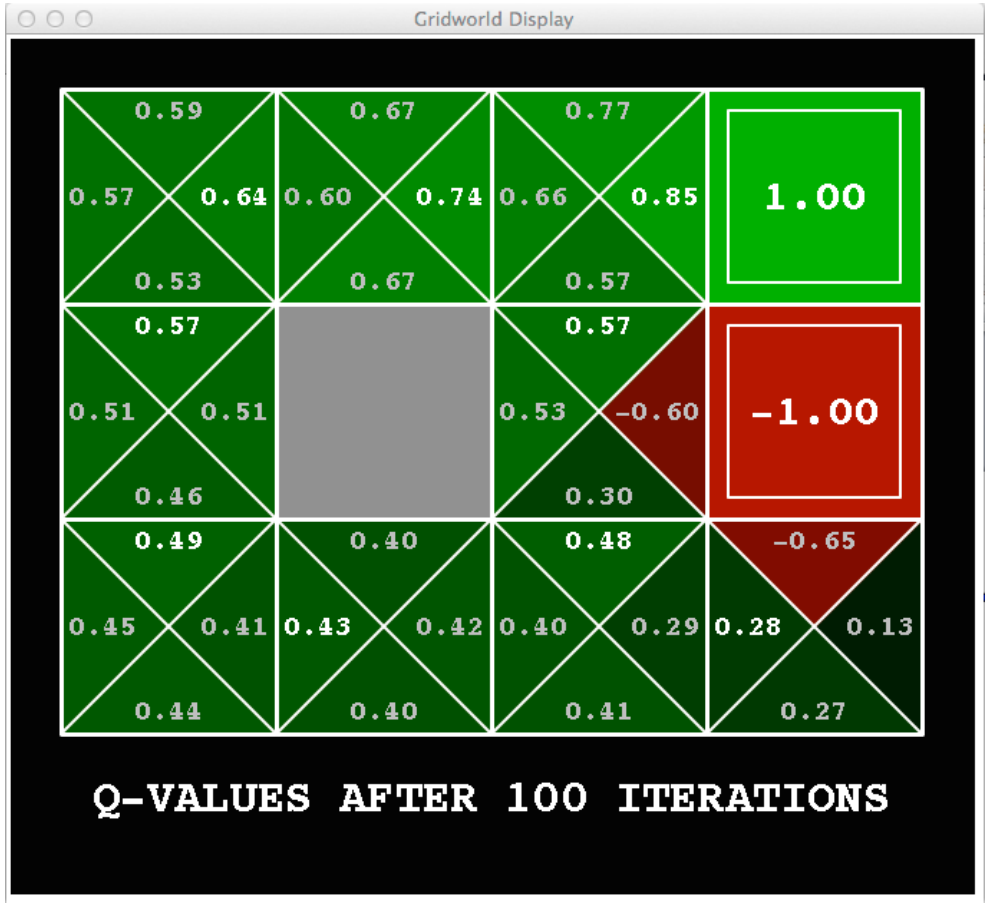


Snapshot of Gridworld V Values



Noise = 0.2
Discount = 0.9
Living reward = 0

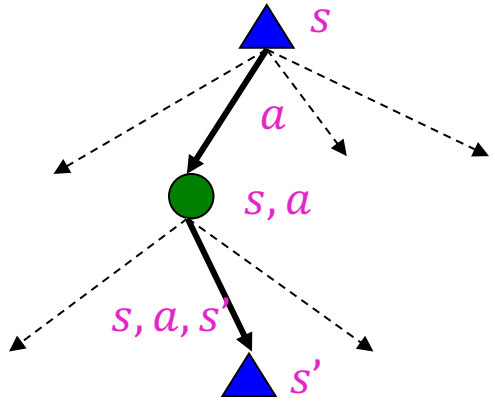
Snapshot of Gridworld Q Values



Noise = 0.2
Discount = 0.9
Living reward = 0

Values of States

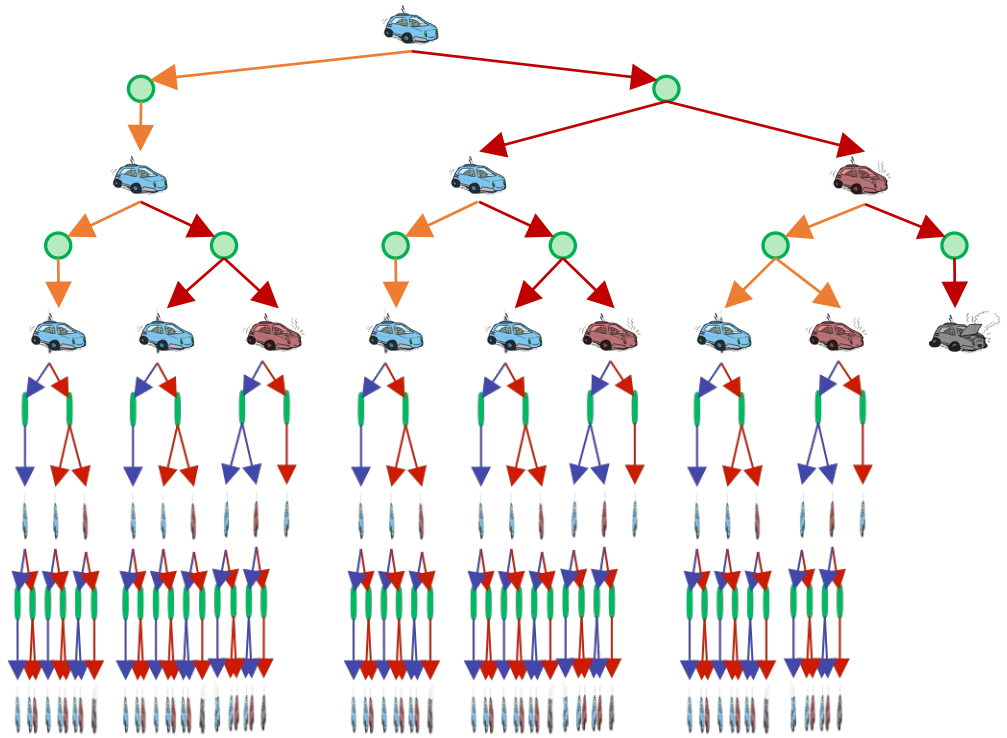
- Fundamental operation: compute the (expectimax) value of a state
  - Expected utility under optimal action
  - Average sum of (discounted) rewards



- Recursive definition of value –
  - Bellman Equations:
  - $V^*(s) = \max_a Q^*(s, a)$
  - $Q^*(s, a) = \sum_{s'} T(s, a, s')[R(s, a, s') + \gamma V^*(s')]$
  - $V^*(s) = \max_a \sum_{s'} T(s, a, s')[R(s, a, s') + \gamma V^*(s')]$

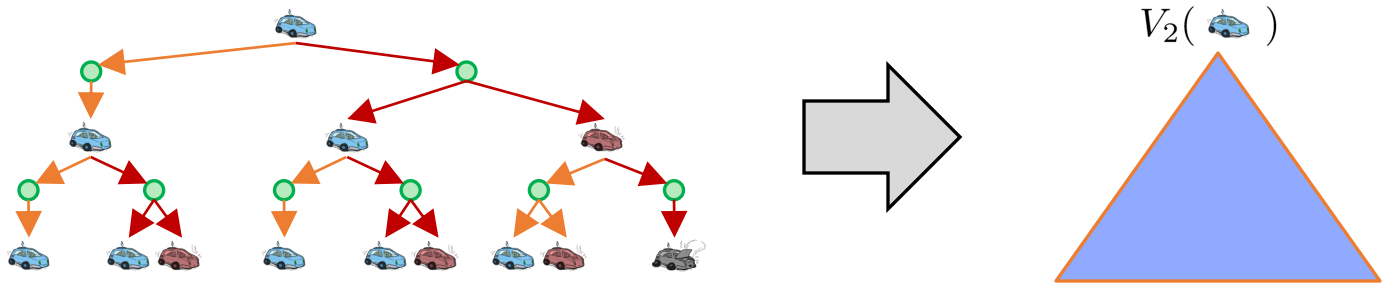
Racing Search Tree

- We're doing way too much work with expectimax!
  - Problem: States are repeated
    - Idea: Only compute needed quantities once
  - Problem: Tree goes on forever
    - Idea: Do a depth-limited computation, but with increasing depths until change is small
- Note: deep parts of the tree eventually don't matter if $\gamma < 1$

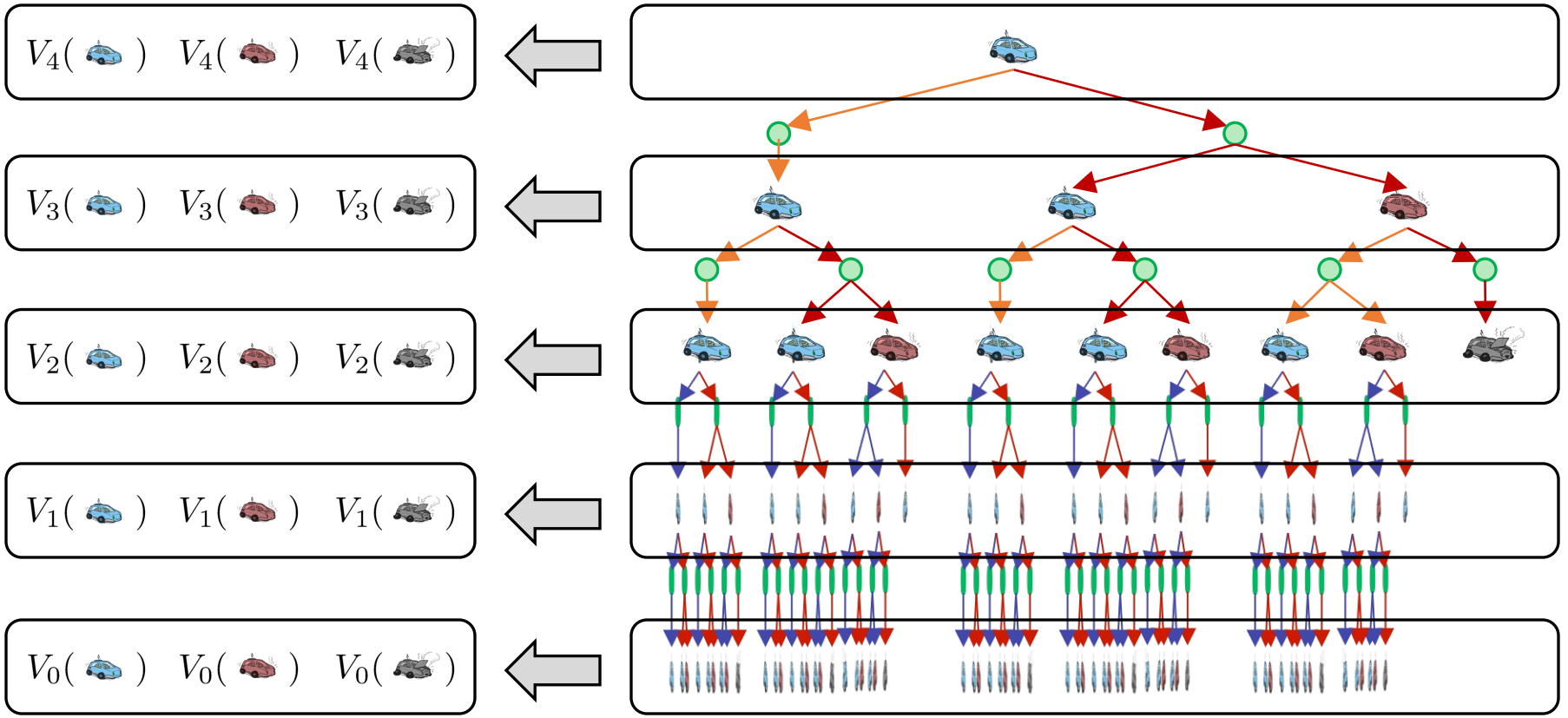


Time-Limited Values

- Key idea: time-limited values
- Define $V_k(s)$ to be the optimal value of s if the game ends in k more time steps
  - Equivalently, it is what a depth-k expectimax would give from s

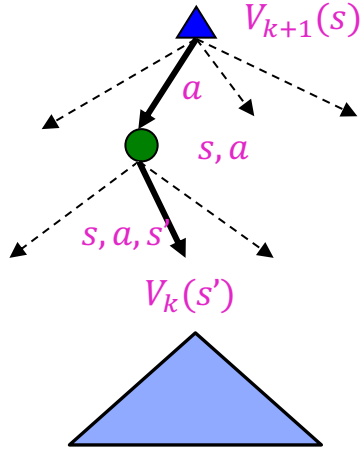


Computing Time-Limited Values

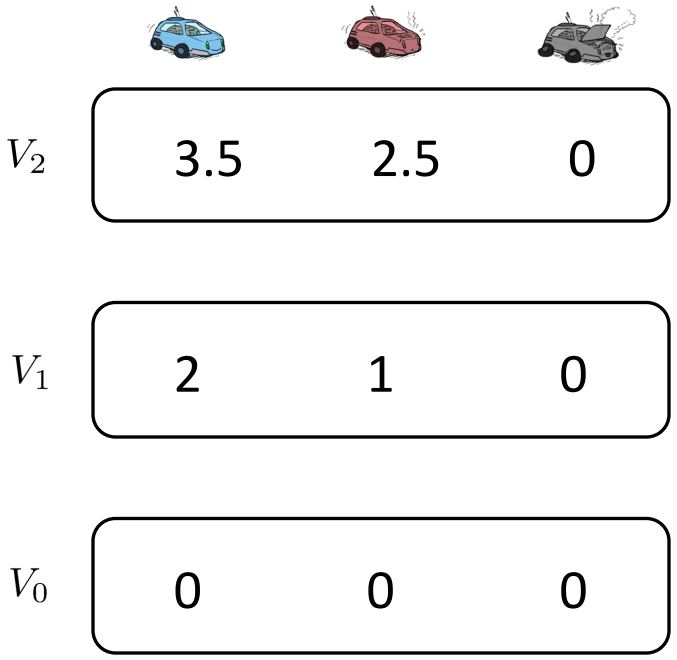


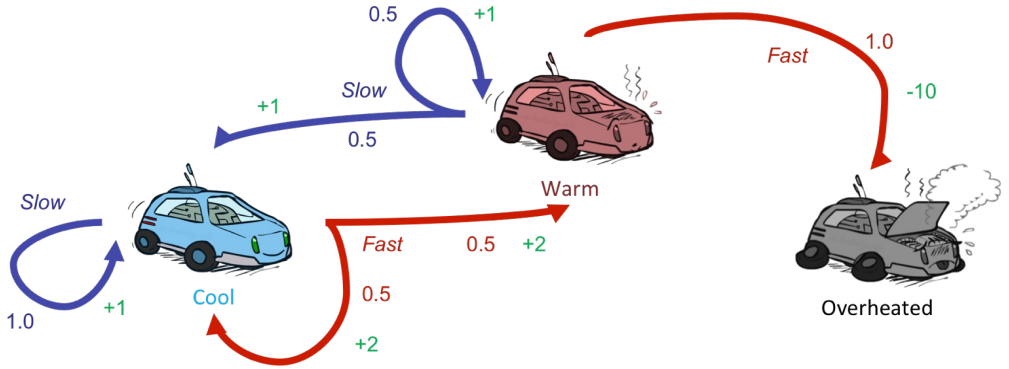
Value Iteration

- Start with $V_0(s) = 0$: no time steps left means an expected reward sum of zero
- Given vector of $V_k(s)$ values, do one ply of expectimax from each state:
  - $V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V_k(s')]$
  - Repeat until convergence



Example: Value Iteration

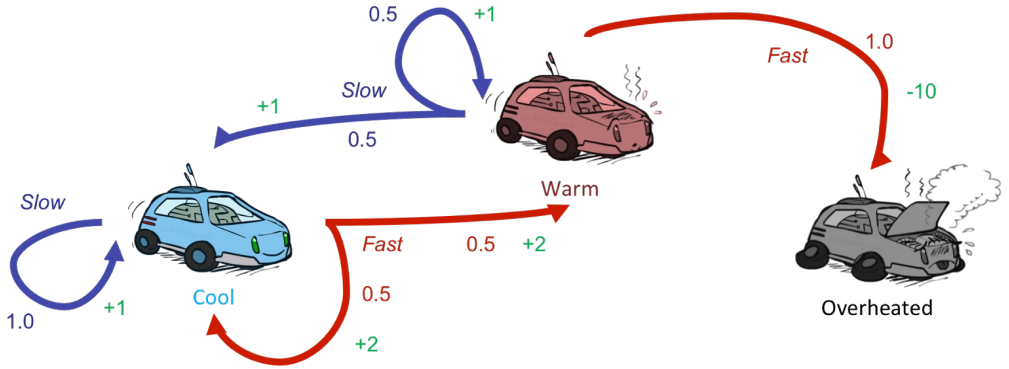




Assume no discount!

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V_k(s')]$$

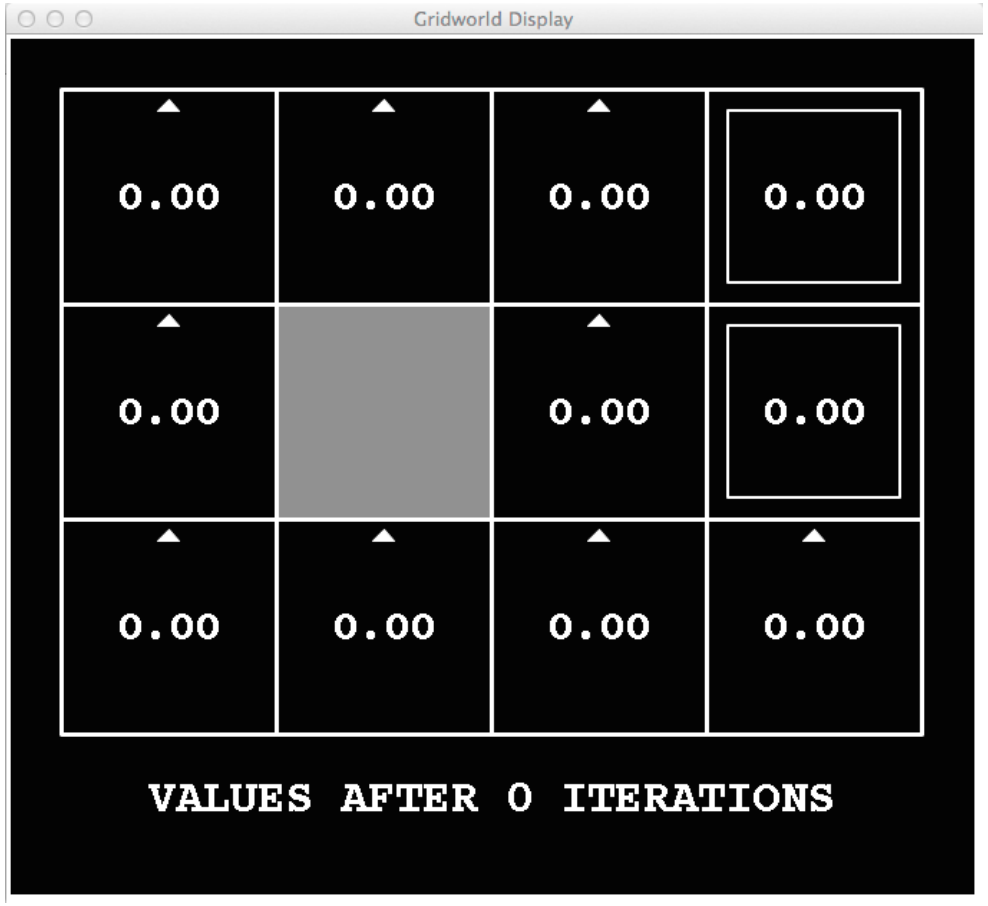
Example: Value Iteration



Assume no discount!

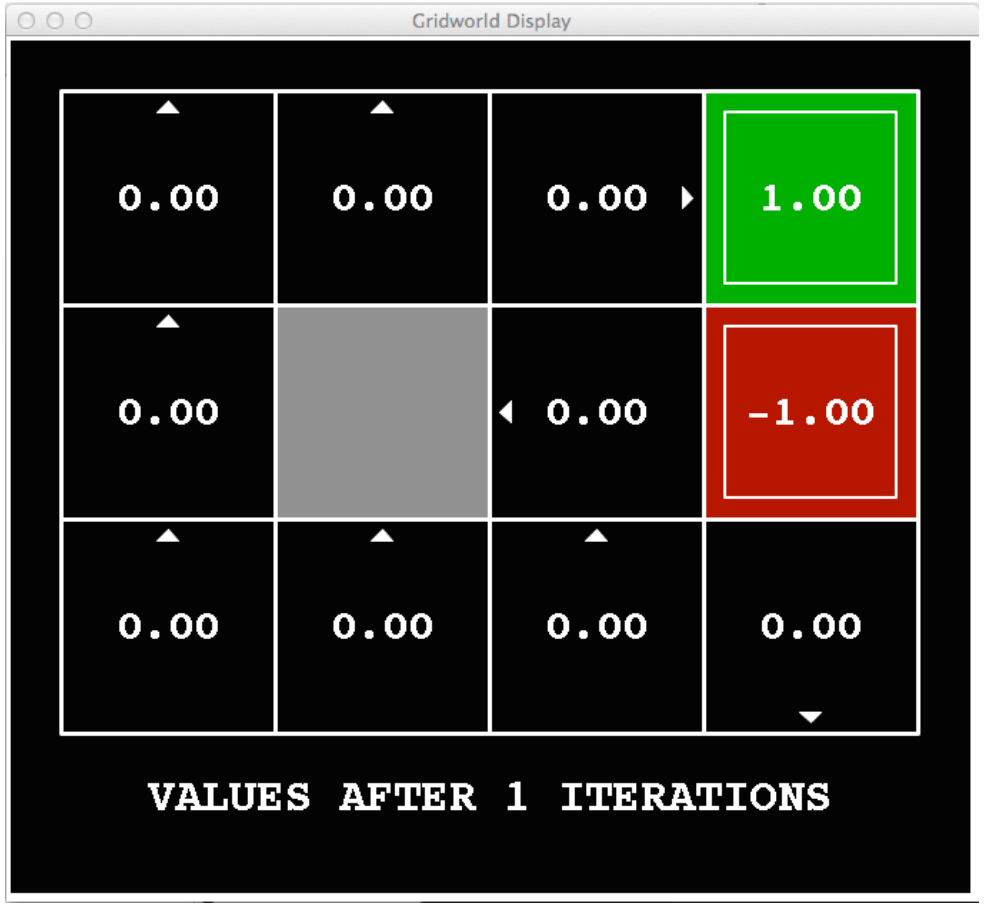
$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V_k(s')]$$

k=0



Noise = 0.2
Discount = 0.9
Living reward = 0

k=1



Noise = 0.2
Discount = 0.9
Living reward = 0

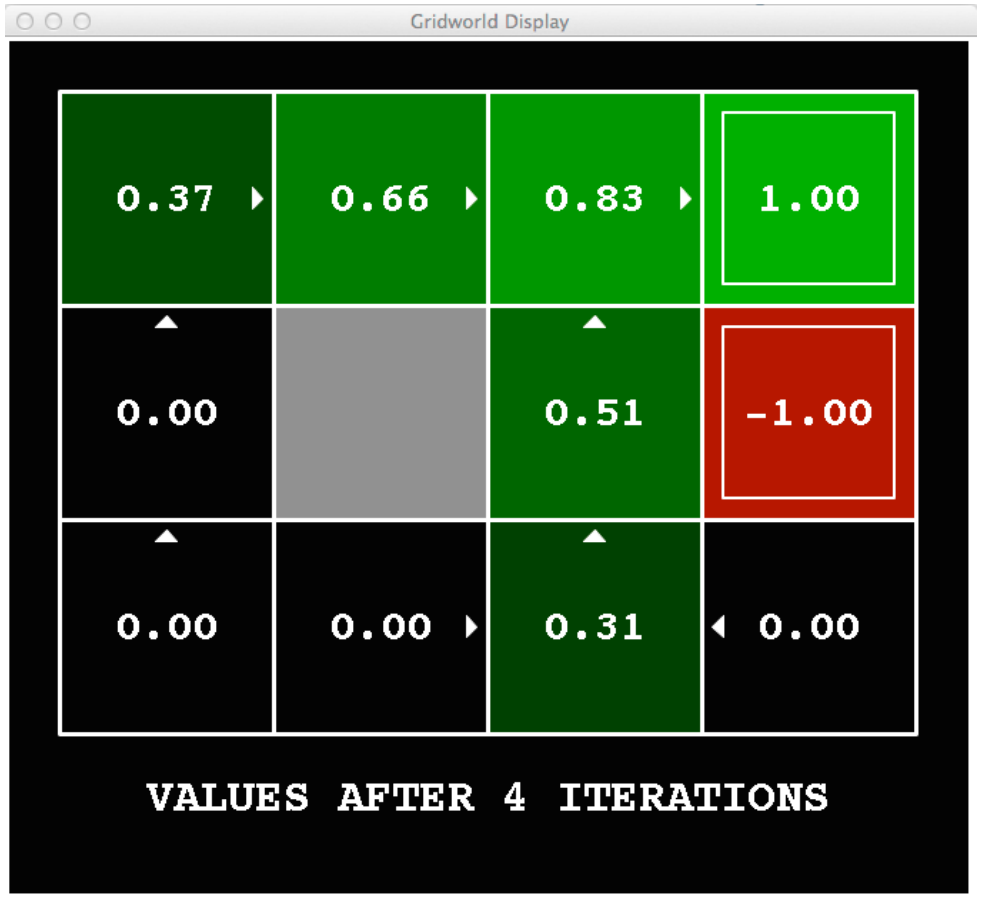
k=2

Noise = 0.2
Discount = 0.9
Living reward = 0

k=3

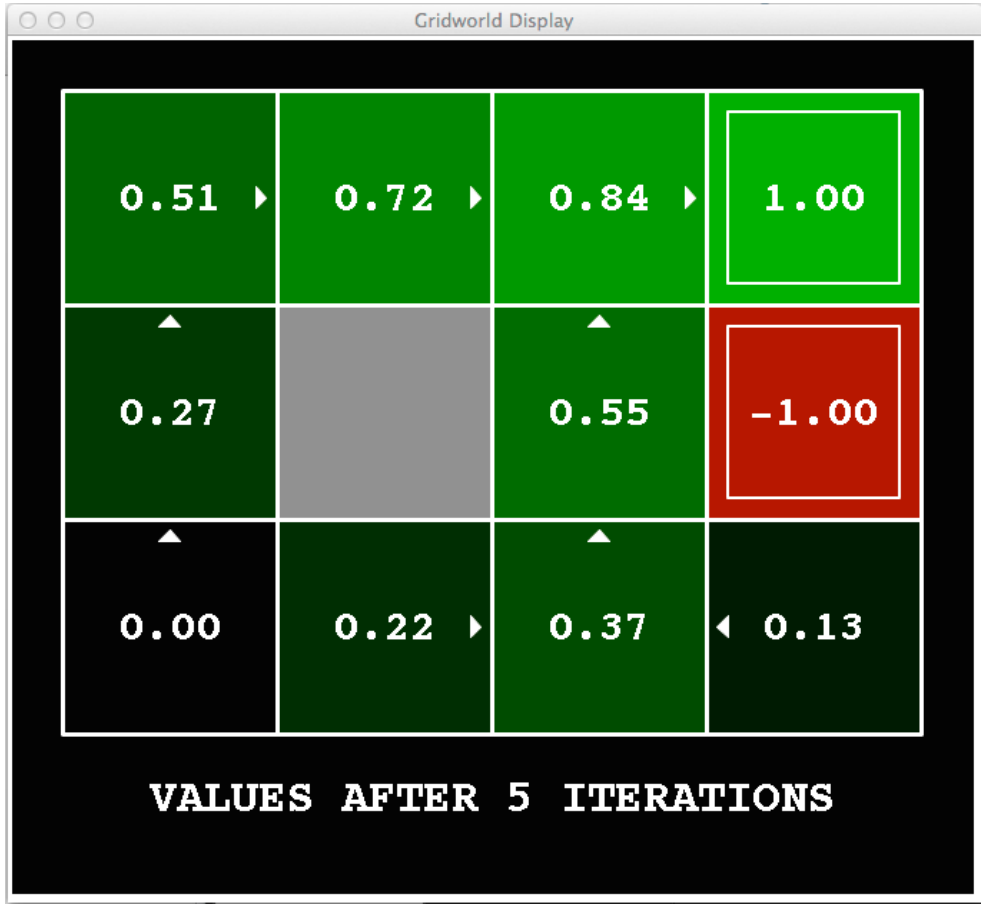
Noise = 0.2
Discount = 0.9
Living reward = 0

k=4



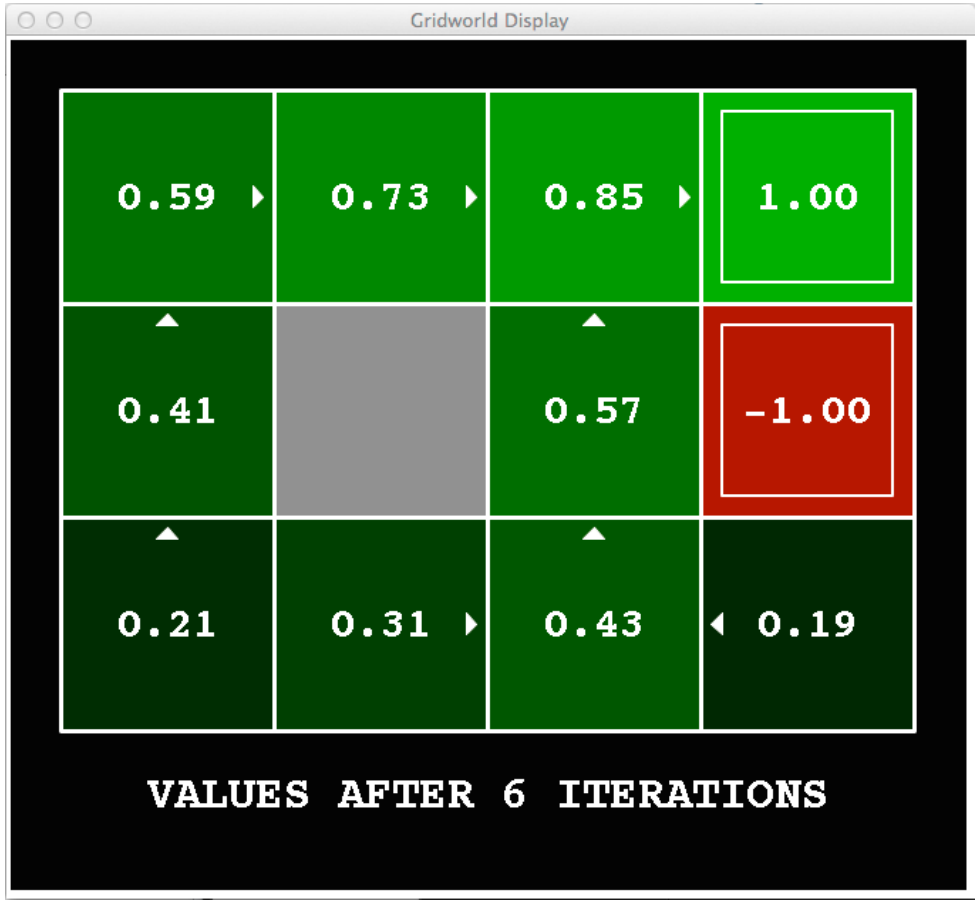
Noise = 0.2
Discount = 0.9
Living reward = 0

k=5



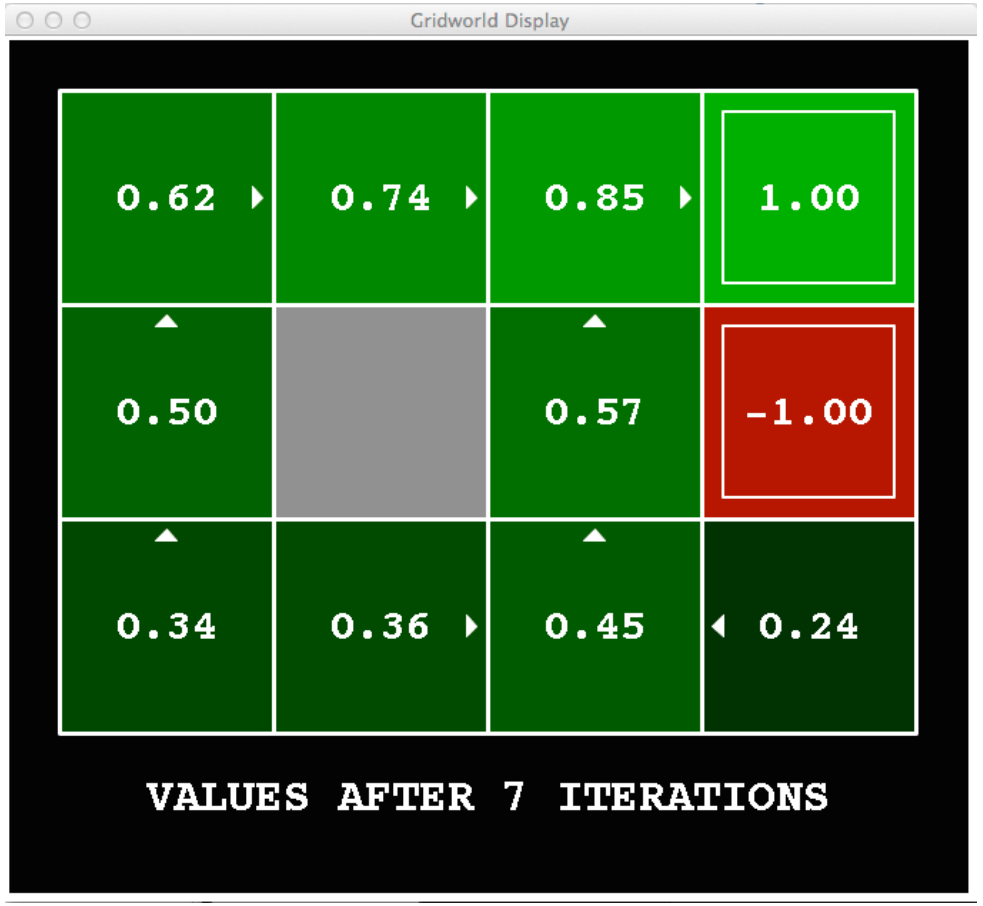
Noise = 0.2
Discount = 0.9
Living reward = 0

k=6



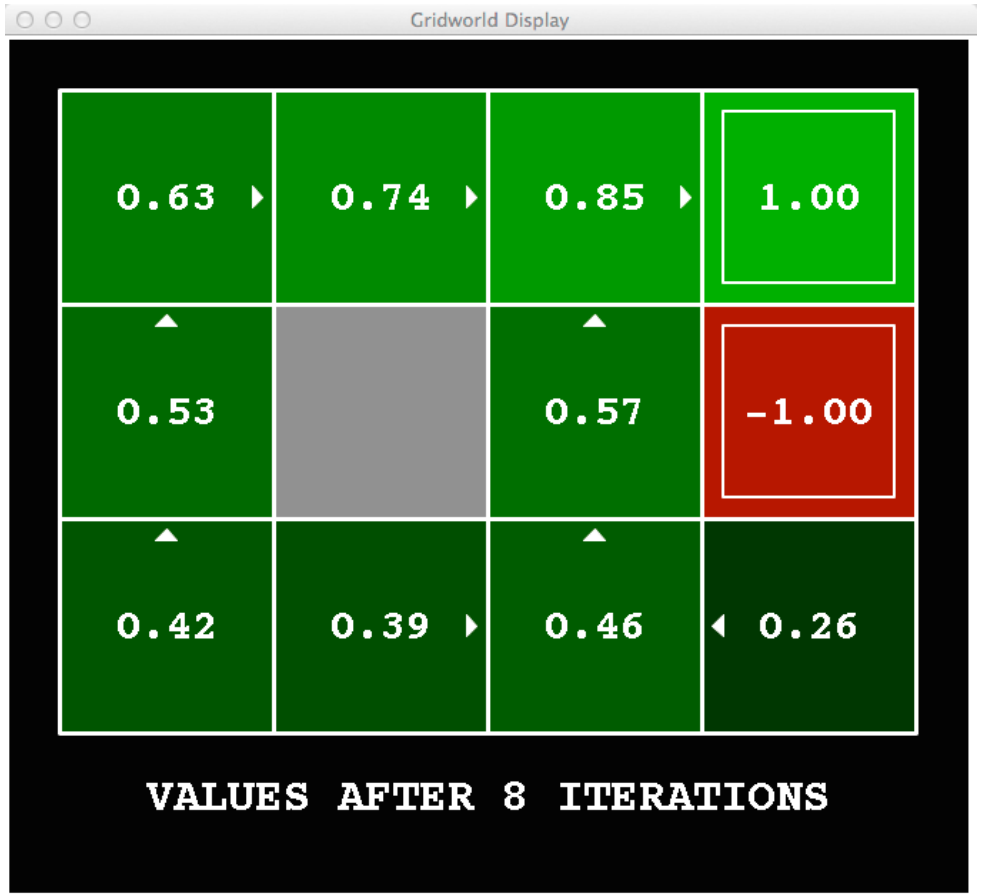
Noise = 0.2
Discount = 0.9
Living reward = 0

k=7



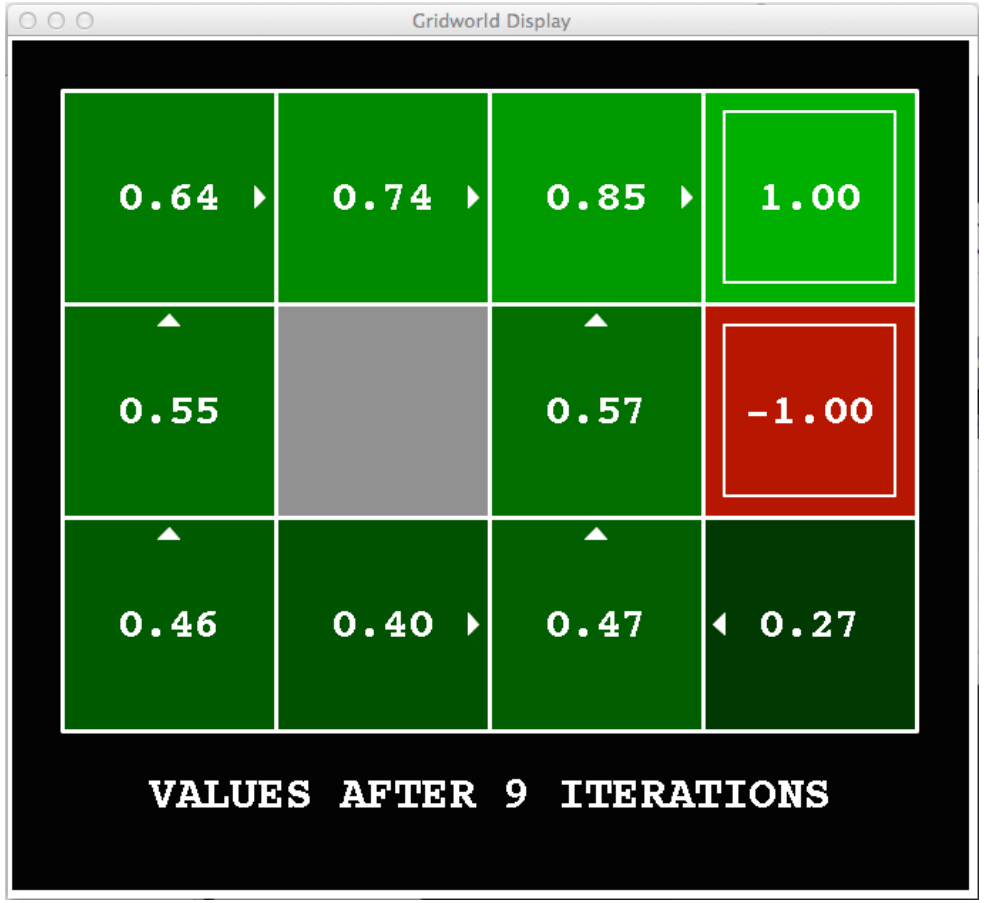
Noise = 0.2
Discount = 0.9
Living reward = 0

k=8



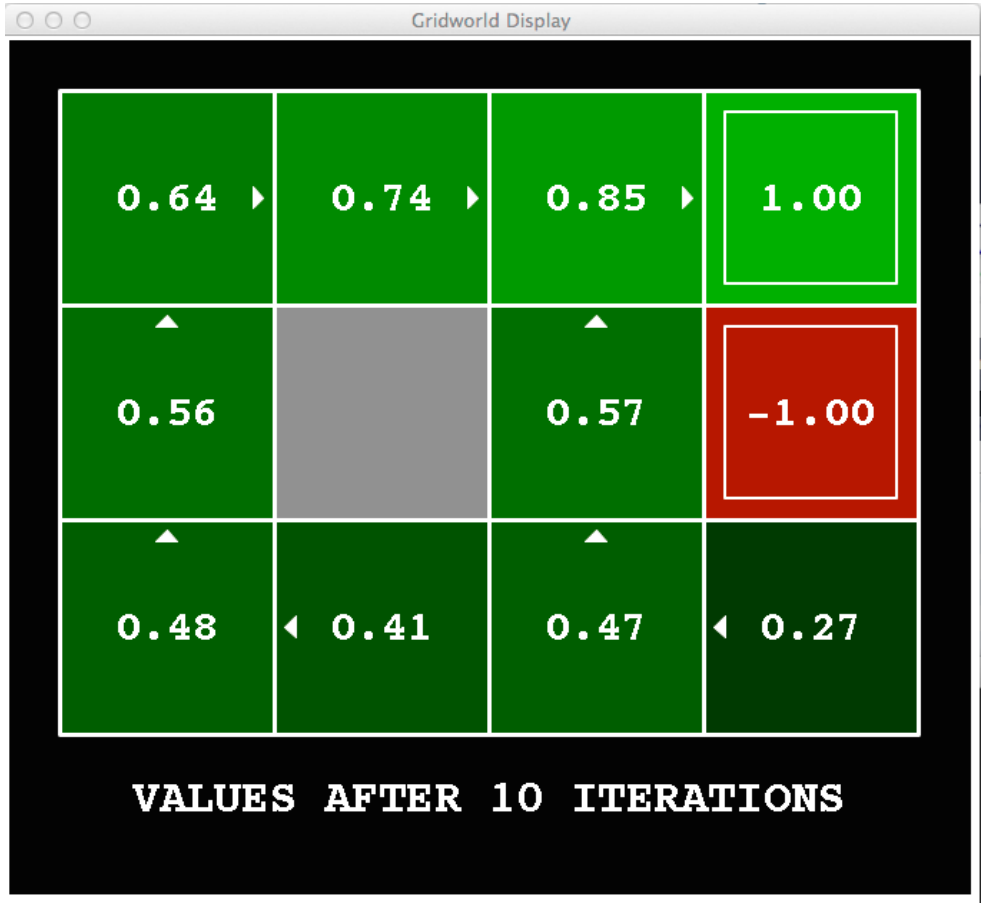
Noise = 0.2
Discount = 0.9
Living reward = 0

k=9



Noise = 0.2
Discount = 0.9
Living reward = 0

k=10



Noise = 0.2
Discount = 0.9
Living reward = 0

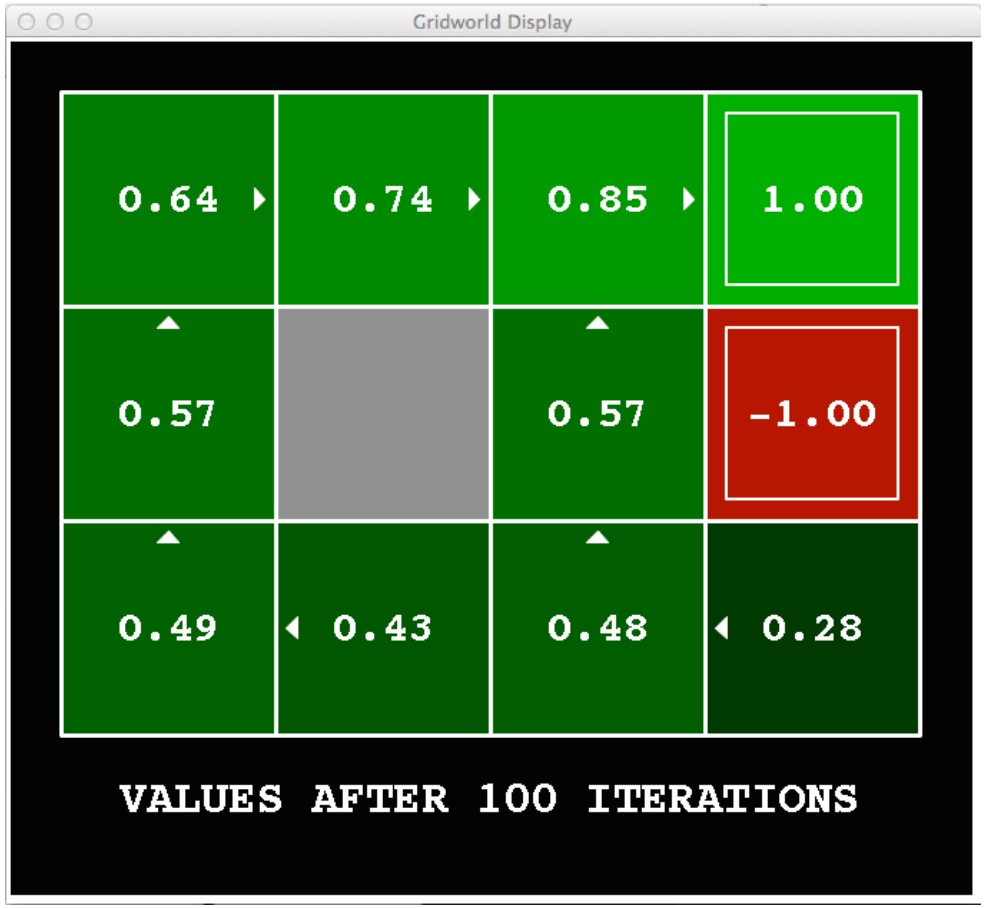
k=11

Noise = 0.2
Discount = 0.9
Living reward = 0

k=12

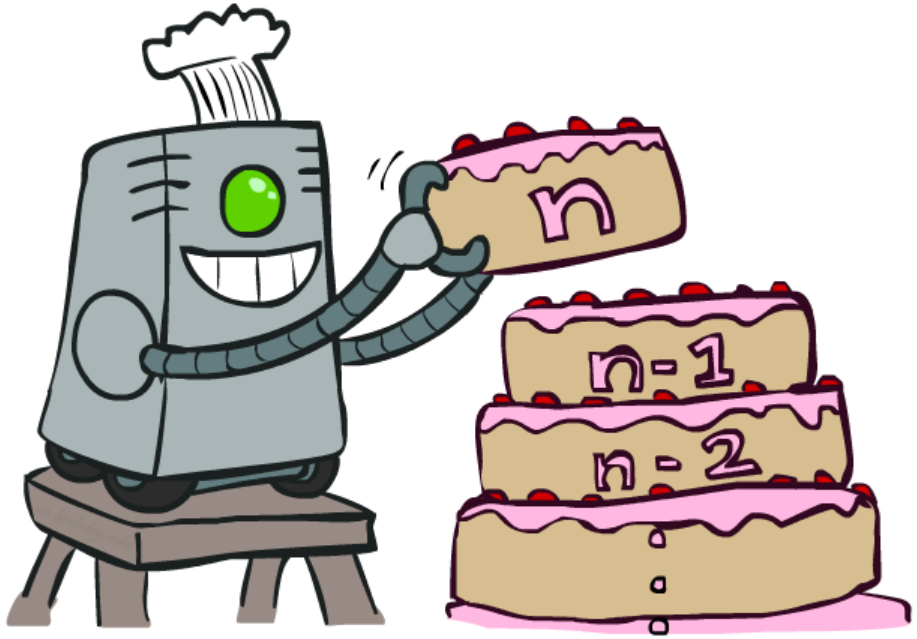
Noise = 0.2
Discount = 0.9
Living reward = 0

k=100



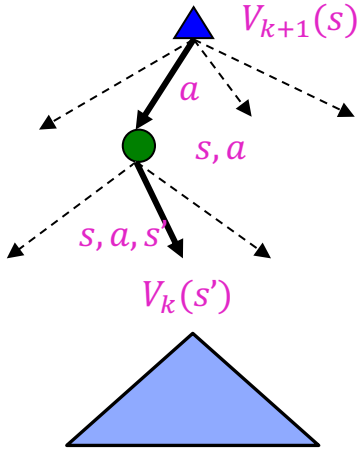
Noise = 0.2
Discount = 0.9
Living reward = 0

Value Iteration



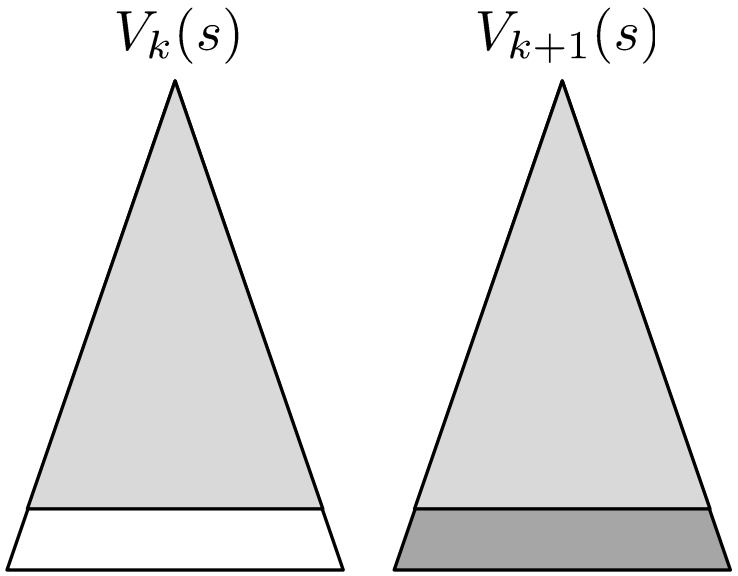
Value Iteration

- Start with $V_0(s) = 0$: no time steps left means an expected reward sum of zero
- Given vector of $V_k(s)$ values, do one ply of expectimax from each state:
  - $V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V_k(s')]$
  - Repeat until convergence
- Complexity of each iteration: $O(S^2 A)$
- Theorem: will converge to unique optimal values
  - Basic idea: approximations get refined towards optimal values
  - Policy may converge long before values do



Convergence*

- How do we know the V_k vectors are going to converge?
- Case 1: If the tree has maximum depth M, then V_M holds the actual untruncated values
- Case 2: If the discount is less than 1
  - Sketch: For any state, V_k and V_{k+1} can be viewed as depth $k+1$ expectimax results in nearly identical search trees
  - The difference is that on the bottom layer, V_{k+1} has actual rewards while V_k has zeros
  - That last layer is at best all R_{MAX}
  - It is at worst R_{MIN}
  - But everything is discounted by γ^k that far out
  - So V_k and V_{k+1} are at most $\gamma^k R_{MAX}$ different
  - So as k increases, the values converge



Convergence of Value Iteration

- View the value function as a point in $|S|$ dimensional space
  - Assume for simplicity finite MDP (finite number of states and actions)
- Define a norm for measuring magnitude of utility vectors - max norm
  - $\|V\| = \sup_s |V(s)|$
- View the Bellman update as an operator B that is applied simultaneously to update the utility of every state.

$$V_{k+1} \leftarrow BV_k$$

Convergence of Value Iteration – Contraction Mapping

Contraction Mapping

- A function that when applied to different inputs in turn produces output values that are closer together by at least a constant factor, than the original inputs
- Example – divide by two

Properties of Contraction Mapping

- Has only one fixed point
- When the function is applied to any argument, the value must get closer to the fixed point, so repeated application of contraction always reaches a fixed point in the limit

Convergence of Value Iteration

- The Bellman update B is a contraction mapping
- Let V_k and V_k' be any two value vectors. Then we have

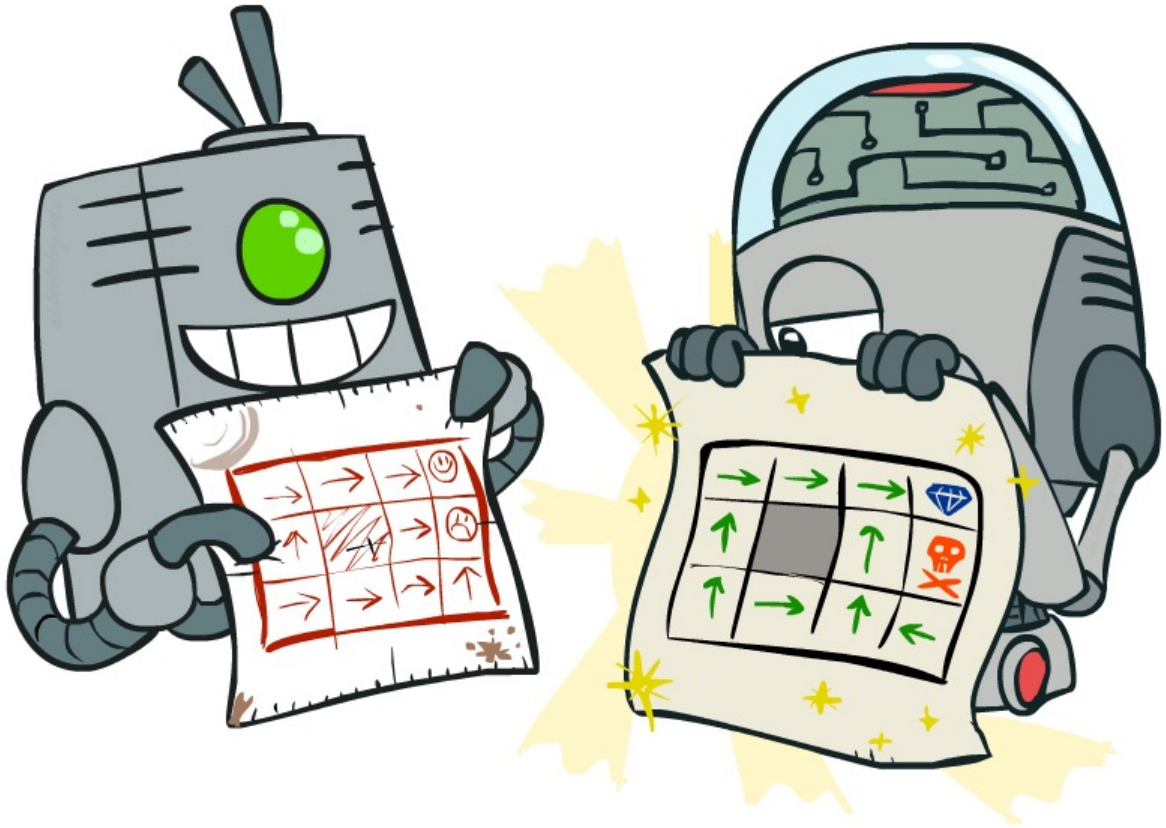
$$\|BV_k - BV_k'\| \leq \gamma \|V_k - V_k'\|$$

- Bellman update is a contraction by a factor of γ on the space of utility vectors
  - Prove it as an exercise – requires principles of vector spaces, Cauchy sequences, convergence of Cauchy sequences, Banach space, and finally Banach fixed point theorem
- The fixed point is V^* - $\mathrm{V}^* \leftarrow BV^*$

Value Iteration

- Rate of convergence
  - $\|BV_k - V^*\| \leq \gamma \|V_k - V^*\|$
  - $\|V_k - V^*\|$ - can be viewed as the error in the estimate of the value V_k
  - Every iteration, the error reduces by a factor of at least γ -resulting in exponential rate of convergence
- How many iterations are required to achieve an error less than ϵ?
  - Maximum initial error - $\|V_0 - V^*\| \leq 2R_m/(1-\gamma)$
  - As error decreases exponentially in γ, we require
    - $\gamma^N 2R_m/(1-\gamma) \leq \epsilon$ implying $N = \left\lceil \log\left(\frac{2R_m}{\epsilon(1-\gamma)}\right) / \log(1/\gamma) \right\rceil$
  - Practically $\|V_k - V\| \leq \epsilon \Rightarrow \|V^{\pi_k} - V\| \leq 2\epsilon\gamma/(1-\gamma)$

Policy Methods



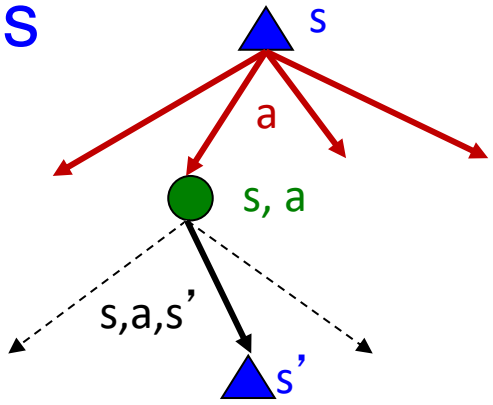
Policy Evaluation
Policy Improvement/Update

Observations with Value Iteration

- Value iteration repeats the Bellman updates:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V_k(s')]$$

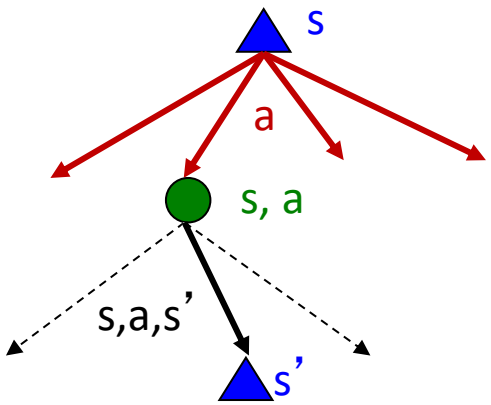
- $O(S^2 A)$ per iteration
- The "max" action at each state rarely changes
- The policy often converges long before the values



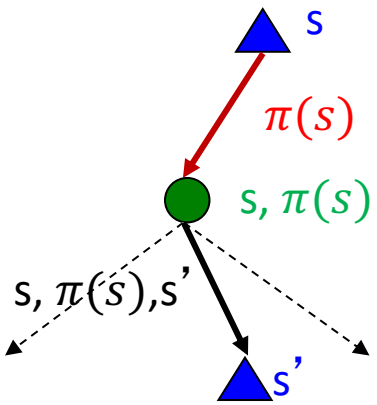
Fixed Policies

- Expectimax trees max over all actions to compute the optimal values
  - If we fixed some policy $\pi(s)$, then the tree would be simpler – only one action per state
  - … though the tree's value would depend on which policy we fixed

Do the optimal action



Do what π says to do



Utilities for a Fixed Policy

- Another basic operation: compute the utility of a state s under a fixed (generally non-optimal) policy
- Define the utility of a state s, under a fixed policy π:
  - $V^{\pi}(s)$ = expected total discounted rewards starting in s and following π
- Recursive relation (one-step look-ahead / Bellman equation):

$$V^{\pi}(s) \leftarrow \sum_{s'} T(s, \pi(s), s')[R(s, \pi(s), s') + \gamma V^{\pi}(s')]$$

Example: Policy Evaluation

Always Go Right

Always Go Forward

Policy Evaluation

$$V^{\pi}(s) \leftarrow \sum_{s'} T(s, \pi(s), s')[R(s, \pi(s), s') + \gamma V^{\pi}(s')]$$

- How do you solve this?
- Note the absence of max operator
  - Results in a linear system of equations.
    - S equations, S unknowns…
- Can be solved using standard linear algebra methods in $O(S^3)$ time.
- Can we do better?

Policy Evaluation (2)

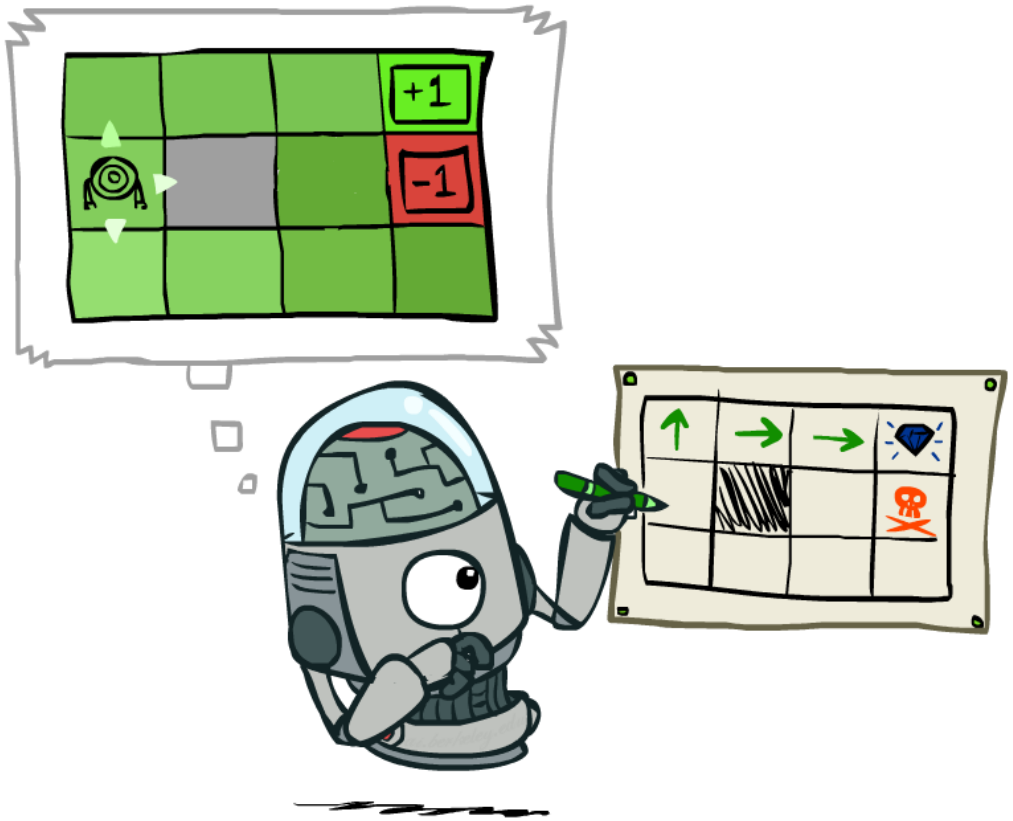
- Turn recursive Bellman equations into updates (like value iteration)

$$V_0^\pi(s) = 0$$

$$V_{k+1}^\pi(s) \leftarrow \sum_{s'} T(s, \pi(s), s')[R(s, \pi(s), s') + \gamma V_k^\pi(s')]$$

- Efficiency: $O(S^2)$ per iteration
- And the resulting algorithm referred to as *Modified policy iteration*

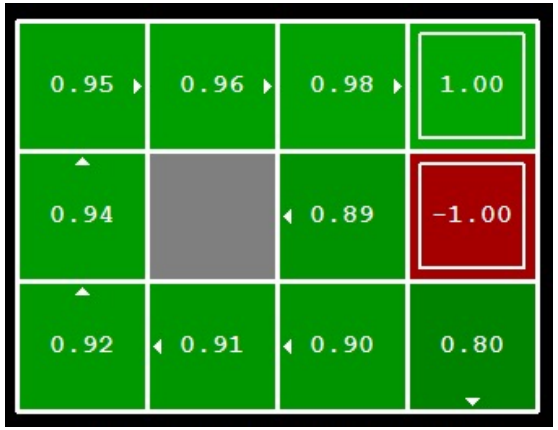
Policy Extraction



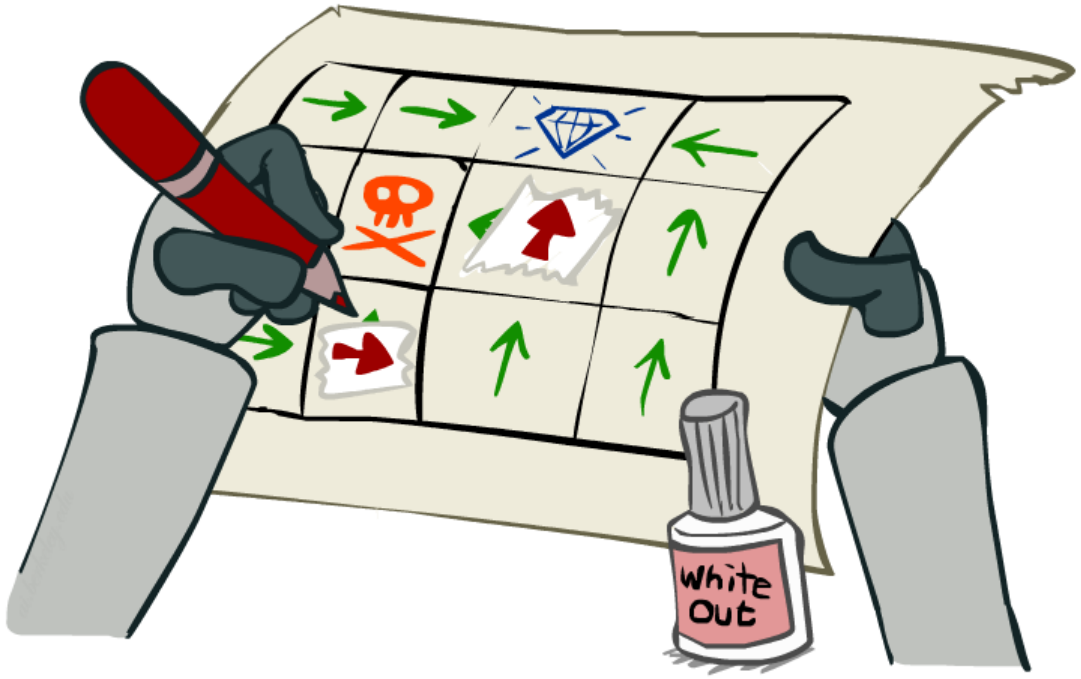
Computing Actions from Values

- Let's imagine we have the optimal values $V^*(s)$
- We need to do a mini-expectimax (one step)
- This is called policy extraction, since it gets the policy implied by the values

$$\pi^*(s) = \arg\max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$



(Modified) Policy Iteration



(Modified) Policy Iteration (2)

- Alternative approach for optimal values:
  - Step 1: Policy evaluation: calculate utilities for some fixed policy (not optimal utilities!) until convergence
  - Step 2: Policy improvement: update policy using one-step look-ahead with resulting converged (but not optimal!) utilities as future values
  - Repeat steps until policy converges.

(Modified) Policy Iteration (3)

- Evaluation: For fixed current policy π, find values with policy evaluation:
  - Iterate until values converge:

$$V_{k+1}^{\pi_i}(s) \leftarrow \sum_{s'} T(s, \pi_i(s), s') \left[R(s, \pi_i(s), s') + \gamma V_k^{\pi_i}(s') \right]$$

- Improvement: For fixed values, get a better policy using policy extraction
  - One-step look-ahead:

$$\pi_{i+1}(s) = \arg\max_a \sum_{s'} T(s, a, s') \left[R(s, a, s') + \gamma V^{\pi_i}(s') \right]$$

Comparison

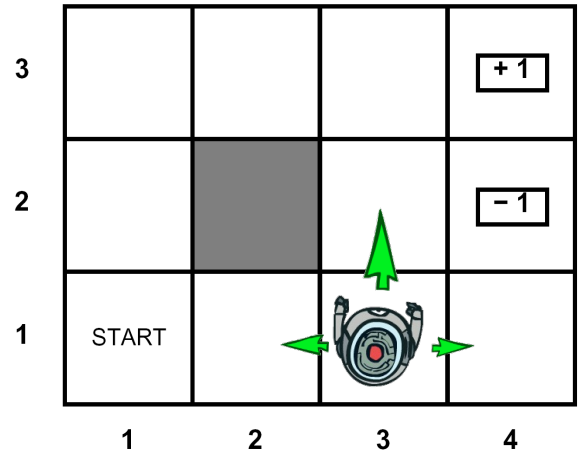
- Both value iteration and policy iteration compute the same thing (all optimal values)
- In value iteration:
  - Every iteration updates both the values and (implicitly) the policy
  - We don't track the policy, but taking the max over actions implicitly recomputes it
- In policy iteration:
  - We do several passes that update utilities with fixed policy (each pass is fast because we consider only one action, not all of them)
  - After the policy is evaluated, a new policy is chosen (slow like a value iteration pass)
  - The new policy will be better (or we're done)

Asynchronous Policy Iteration

- Not necessary to update utilities and policies for each state at once.
- Pick any subset of states and apply the iterative procedures
- Under certain conditions on the initial policy and utility function, this is guaranteed to converge to an optimal policy

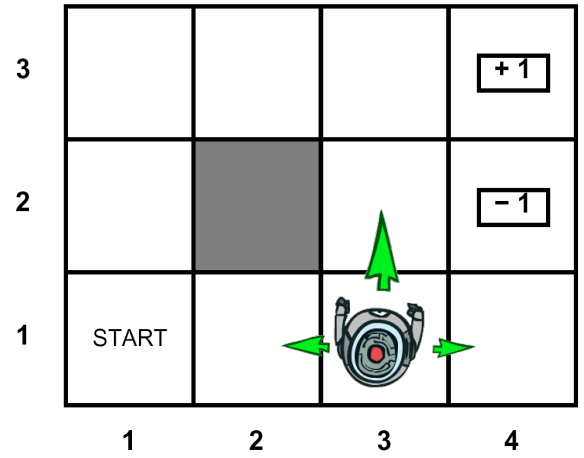
Partially Observable MDPs

- All the elements of an MDP
  - S, A, P, R
- And additional elements
  - Sensor model - $P(e|s)$ - specifies the probability of perceiving the evidence e in state s
- Example – grid world
  - Instead of knowing the exact location
  - Noisy count of adjacent walls



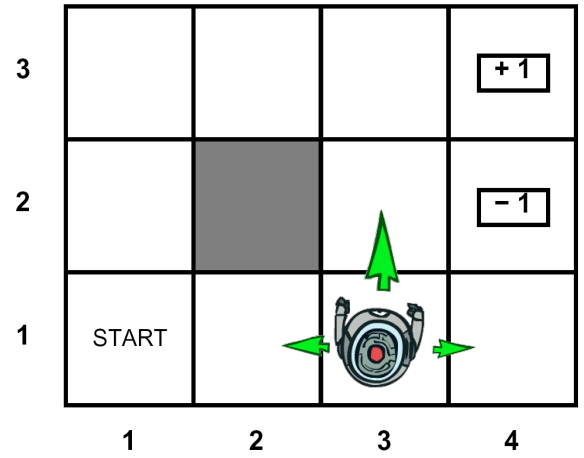
POMDP – Belief State (1)

- Belief state b is a probability distribution over all possible states
  - $b(s)$ – probability assigned to state s by the belief state b



POMDP – Belief State (2)

- Belief state b is a probability distribution over all possible states
  - $b(s)$ – probability assigned to state s by the belief state b
- Agent calculates the new belief state from the previous belief state, the action performed, and the new evidence
  - $b'(s') = \alpha P(e|s') \sum_s P(s'|s,a) b(s)$



POMDP Optimal Policy

- The optimal action depends only on the agent's current belief state
  - $\pi^*: B \rightarrow A$
  - It does not depend on the actual state - s
- Behavior of the agent
  - Execute the optimal action associated with the current belief state
  - Obtain the percept/sensory information
  - Update the belief state (using previous belief state, action performed, and the percept received)

POMDP – Belief state update

- Probability of perceiving e, given that a was performed starting in belief state b
  - $P(e|a,b)$

POMDP – Belief state update

- Probability of perceiving e, given that a was performed starting in belief state b
  - $P(e|a,b) = \sum_{s'} P(e|a,s',b)P(s'|a,b)$
  - $= \sum_{s'} P(e|s') \sum_{s} P(s'|s,a)b(s)$
- Now let us define the probability of reaching b' from b, given action a
  - $P(b'|a,b) =$

POMDP – Belief state update

- Probability of perceiving e, given that a was performed starting in belief state b
  - $P(e|a,b) = \sum_{s'} P(e|a,s',b)P(s'|a,b)$
  - $= \sum_{s'} P(e|s') \sum_{s} P(s'|s,a)b(s)$
- Now let us define the probability of reaching b' from b, given action a
  - $P(b'|a,b) = \sum_{e} P(b'|e,a,b)P(e|a,b)$
  - $= \sum_{e} P(b'|e,a,b) \sum_{s'} P(e|s') \sum_{s} P(s'|s,a)b(s)$
  - $P(b'|e,a,b) = 1$, if b' is reachable from b
- Reward function for belief states

POMDP – Belief state update

- Probability of perceiving e, given that a was performed starting in belief state b
  - $P(e|a,b) = \sum_{s'} P(e|a,s',b)P(s'|a,b)$
  - $= \sum_{s'} P(e|s') \sum_{s} P(s'|s,a)b(s)$
- Now let us define the probability of reaching b' from b, given action a
  - $P(b'|a,b) = \sum_{e} P(b'|e,a,b)P(e|a,b)$
  - $= \sum_{e} P(b'|e,a,b) \sum_{s'} P(e|s') \sum_{s} P(s'|s,a)b(s)$
  - $P(b'|e,a,b) = 1$, if b' is reachable from b'
- Reward function for belief states
  - $\rho(b) = \sum_{s} b(s)R(s)$

POMDP to MDP

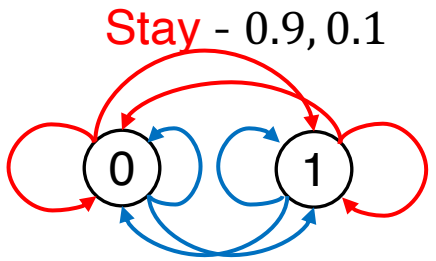
- Given, $P(b'|a,b)$ and $\rho(b)$ the POMDP is transformed into an MDP.
- The optimal policy for this transformed MDP is also an optimal policy for the original POMDP
- This is great! But wait… is there a problem?

POMDP to MDP

- Given, $P(b'|a,b)$ and $\rho(b)$ the POMDP is transformed into an MDP.
- The optimal policy for this transformed MDP is also an optimal policy for the original POMDP
- This is great! But wait… is there a problem?
- The resulting MDP is continuous!

Value Iteration for POMDP

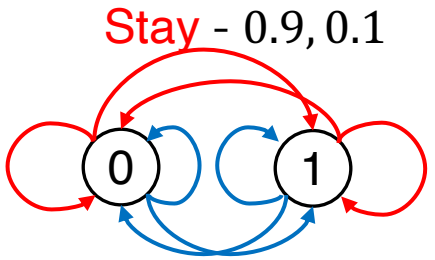
- Consider a 2-state world
  - $\gamma = 1$
  - Sensor reports the correct state with probability 0.6
  - Obviously, the agent should Stay when it thinks it is in state 1 and Go when it thinks it is in state 0
- What is the belief state?



$R(0) = 0$ $R(1) = 1$

Value Iteration for POMDP (1)

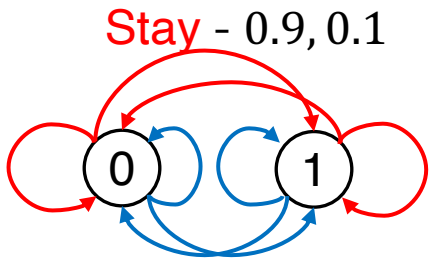
- Consider a 2-state world
  - $\gamma = 1$
  - Sensor reports the correct state with probability 0.6
  - Obviously the agent should Stay when it thinks it is in state 1 and Go when it thinks it is in state 0
- What is the belief state?
  - $b(1) \in [0,1]$ and $b(0) = 1 - b(1)$
  - Essentially a 1D representation.



$R(0) = 0$ $R(1) = 1$

Value Iteration for POMDP (2)

- Consider a one-step plan, i.e., the agent performs the only single action
  - $[stay]$ or $[go]$
- Let the reward obtained after the one step be $\alpha_a(s)$
  - $\alpha_{stay}(0) =$
  - $\alpha_{stay}(1) =$
  - $\alpha_{go}(0) =$
  - $\alpha_{go}(1) =$



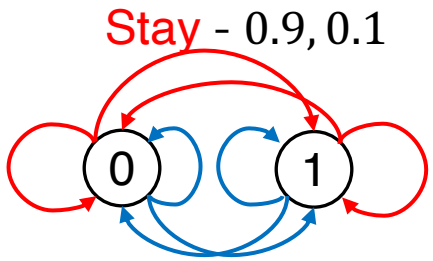
$R(0) = 0 \qquad R(1) = 1$

$\gamma = 1$

$Noise = 0.6$

Value Iteration for POMDP (2)

- Consider a one-step plan, i.e., the agent performs the only single action
  - $[stay]$ or $[go]$
- Let the reward obtained after the one step be $\alpha_a(s)$
  - $\alpha_{stay}(0) = R(0) + \gamma(0.9R(0) + 0.1R(1)) = 0.1$
  - $\alpha_{stay}(1) = R(1) + \gamma(0.9R(1) + 0.1R(0)) = 1.9$
  - $\alpha_{go}(0) = R(0) + \gamma(0.9R(1) + 0.1R(0)) = 0.9$
  - $\alpha_{go}(1) = R(1) + \gamma(0.9R(0) + 0.1R(1)) = 1.1$
  - These were obtained assuming that the agent is certain about the state s



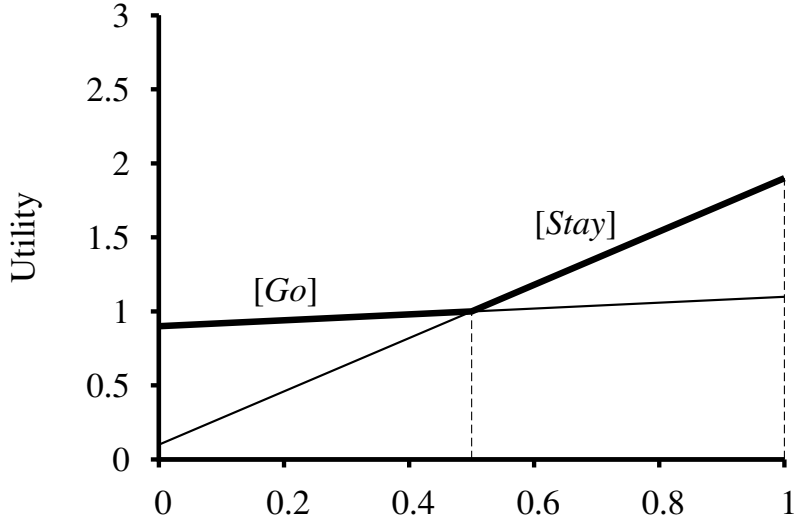
$R(0) = 0 \qquad R(1) = 1$

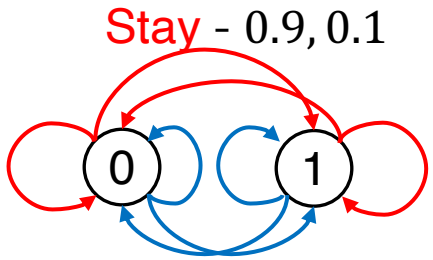
$\gamma = 1$

$Noise = 0.6$

Value Iteration for POMDP (3)

- How will you extend the expected reward obtained for the one step action, if the state is a belief state?
  - $\sum_s b(s)\alpha_p(s)$
  - Value function - maximum expected utility
  - One-step policy
    - $stay\ if\ b(1) > 0.5$
    - $else\ go$





$R(0) = 0 \qquad R(1) = 1$

$\gamma = 1$

$Noise = 0.6$

Single step

$[stay]$ or $[go]$

$\alpha_{stay}(0) = 0.1$

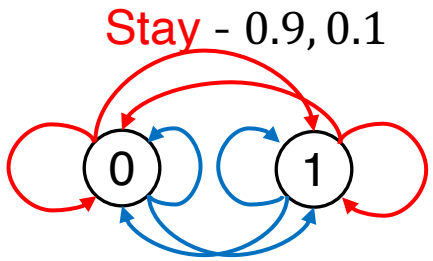
$\alpha_{stay}(1) = 1.9$

$\alpha_{go}(0) = 0.9$

$\alpha_{go}(1) = 1.1$

Value Iteration for POMDP (3)

- Given conditional plan p for depth 1 in each physical state, compute the conditional plans of depth 2.
  - Consider each possible first action
  - Each possible subsequent percept
  - Each way of choosing a depth 1 plan for each percept:
    - [stay, 0, stay]
    - [stay, 0, go]
    - [stay, 1 stay]
    - [stay, 1 go]
    - [go, 0, stay]…



$R(0) = 0 \qquad R(1) = 1$

$\gamma = 1$

$Noise = 0.6$

Single step

$[stay]$ or $[go]$

$\alpha_{stay}(0) = 0.1$

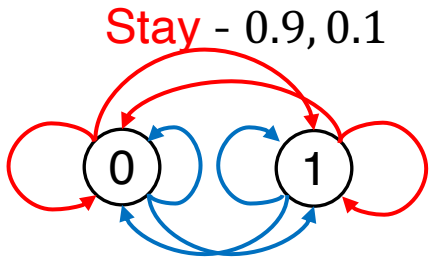
$\alpha_{stay}(1) = 1.9$

$\alpha_{go}(0) = 0.9$

$\alpha_{go}(1) = 1.1$

Value Iteration for POMDP (3)

- Given conditional plan p for depth 1 in each physical state, compute the conditional plans of depth 2.
  - Consider each possible first action
  - Each possible subsequent percept
  - Each way of choosing a depth 1 plan for each percept:
    - [stay, 0, stay]
    - [stay, 0, go]
    - [stay, 1 stay]
    - [stay, 1 go]
    - [go, 0, stay]…



$R(0) = 0 \qquad R(1) = 1$

$\gamma = 1$

$Noise = 0.6$

Single step

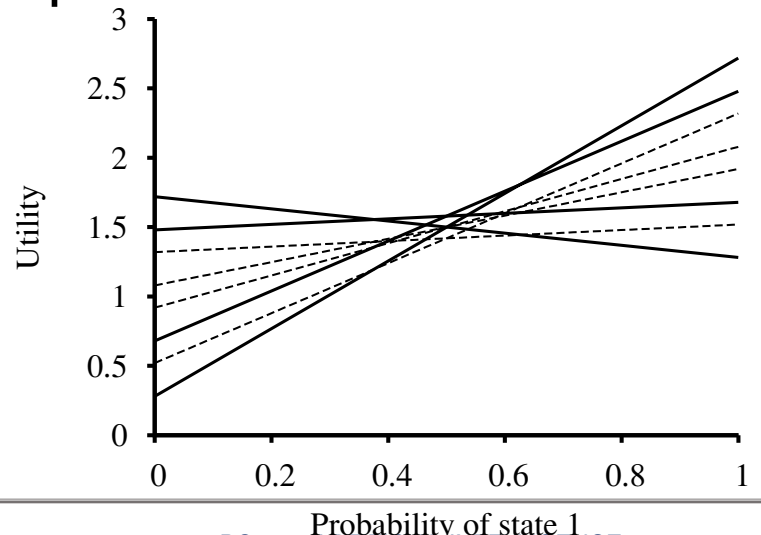
$[stay]$ or $[go]$

$\alpha_{stay}(0) = 0.1$

$\alpha_{stay}(1) = 1.9$

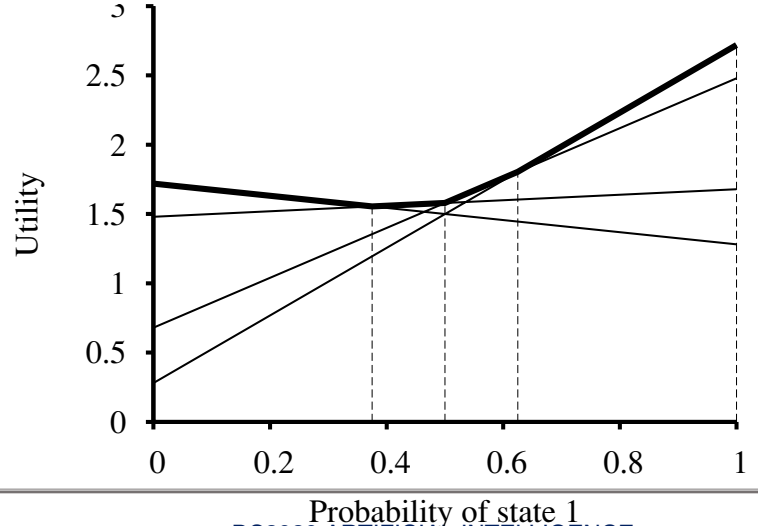
$\alpha_{go}(0) = 0.9$

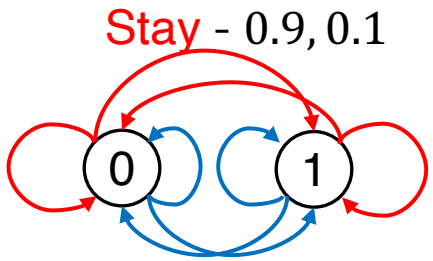
$\alpha_{go}(1) = 1.1$



Value Iteration for POMDP (3)

- Given conditional plan p for depth 1 in each physical state, compute the conditional plans of depth 2.
  - Consider each possible first action
  - Each possible subsequent percept
  - Each way of choosing a depth 1 plan for each percept:
    - [stay, 0, stay]
    - [stay, 0, go]
    - [stay, 1 stay]
    - [stay, 1 go]
    - [go, 0, stay]…





$R(0) = 0 \qquad R(1) = 1$

$\gamma = 1$

$Noise = 0.6$

Single step

$[stay]$ or $[go]$

$\alpha_{stay}(0) = 0.1$

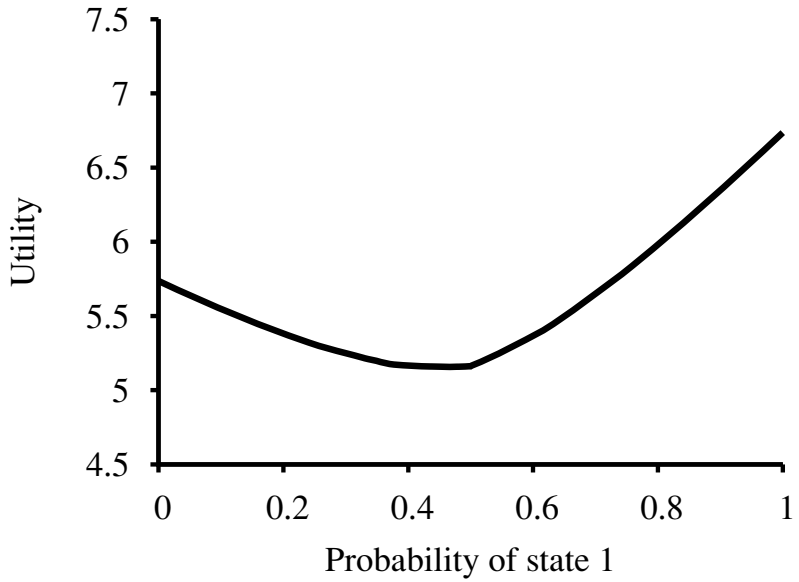
$\alpha_{stay}(1) = 1.9$

$\alpha_{go}(0) = 0.9$

$\alpha_{go}(1) = 1.1$

Value Iteration for POMDP

- After 8 steps



Value Iteration for POMDP

- $V_0(b) = 0$
- $V_k(b) = \max_a [R(b,a) + \gamma \sum_e P(e|b,a) V_{k-1}(b')]$
  - Where $b' = SE(b,a,e)$
  - $R(b,a) = \sum_s b(s) R(s,a)$

- $V_k(b) = \max_{\alpha \in \Gamma_k} \sum_s b(s)\alpha(s)$
  - Piece-wise linear and convex
  - $\alpha(s) = R(s) + \gamma(\sum_{s'} P(s'|s,a) \sum_e P(e|s')\alpha'(s'))$

Summary: MDP Algorithms

- MDPs
- Optimal Policies
- Value Iteration
- Policy Evaluation
- Policy Iteration
- Partially Observable MDP (POMDPS)
- How to learn the MDPs?
  - Reinforcement Learning